Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Программирование графических процессоров

Лабораторная работа №2

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2024

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
1. 4
2. 6
3. 7
Вывод: 8
Листинг 9
1. 9
main.cu (gpu) 9
main.cpp (cpu) 10
2. 12
main.cpp (CPU) 12
3. 14
main.cu (GPU) 14

Задание:

1. Определить при какой длине векторов имеет смысл распараллеливать операцию сложения, используя потоки CPU или GPU.

2. Определить оптимальное количество потоков POSIX для распараллеливания.

3. Определить зависимость времени выполнения операции сложения на GPU от длины векторов (выбирать количество нитей равным длине вектора).

Цель: начальное знакомство с распараллеливанием кода на GPU .

Ход работы:

Number of CUDA devices: 1

Device 0: NVIDIA GeForce GTX 1060 6GB
Compute capability: 6.1
Total global memory: 2147221504 bytes
Total constant memory: 65536 bytes
Shared memory per block: 49152 bytes
Warp size: 32
Max threads per block: 1024
Max threads per multiprocessor: 2048
Max grid size: (2147483647, 65535, 65535)
Max block size: (1024, 1024, 64)
Clock rate: 1771500 kHz
Memory clock rate: 4004000 kHz
Memory bus width: 192 bits
L2 cache size: 1572864 bytes
Max 1D texture size: 131072
Max 2D texture size: (131072, 65536)
Max 3D texture size: (16384, 16384, 16384)
ECC enabled: No
Unified addressing: Yes

1.

Для того, чтобы определить при какой длине векторов имеет смысл распараллеливать операцию сложения, используя GPU напишем программу, которая будет выполнять сложение векторов сначала с количеством блоков = 1 и количеством нитей = 1, затем с количеством блоков = (arraySize + blockSize - 1) / blockSize и количеством нитей в блоке = 512.

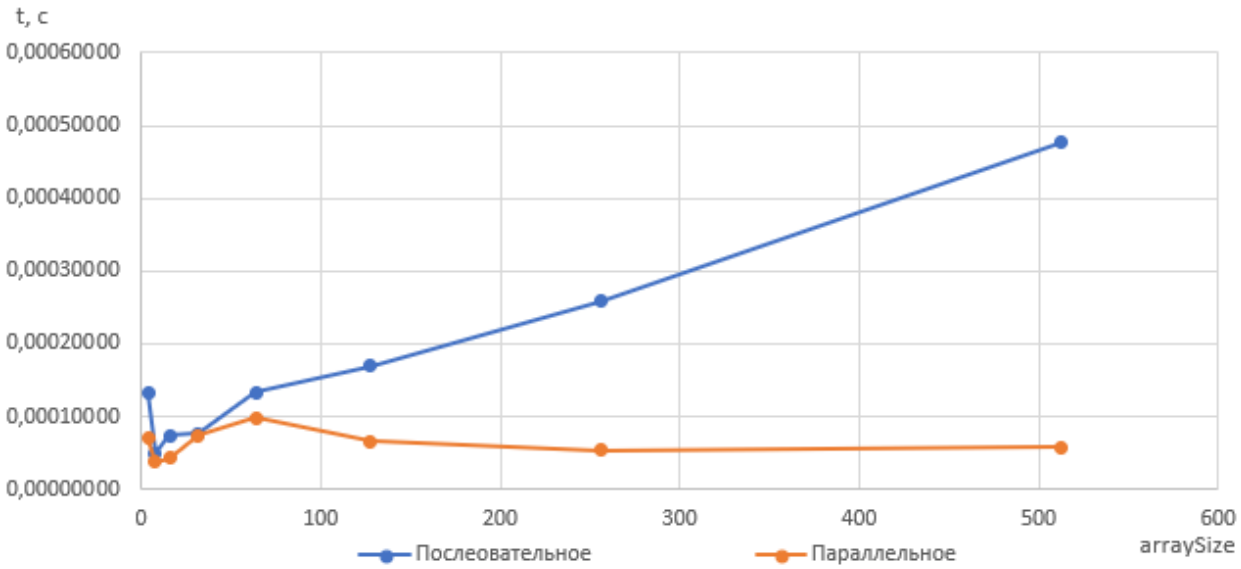


Рис. 1 – Зависимость времени выполнения последовательного (1 блок 1 нить) и параллельного сложения векторов от их размера на GPU.

Из полученных данных можно сделать вывод, что параллельное сложение векторов на GPU происходит быстрее, чем последовательное при любой длине векторов. Преимущество по времени более чем в 2 раза наблюдается при длине векторов больше 128.

Для того, чтобы определить при какой длине векторов имеет смысл распараллеливать операцию сложения, используя CPU напишем программу, которая будет выполнять сложение векторов на 1 и на 4 потоках.

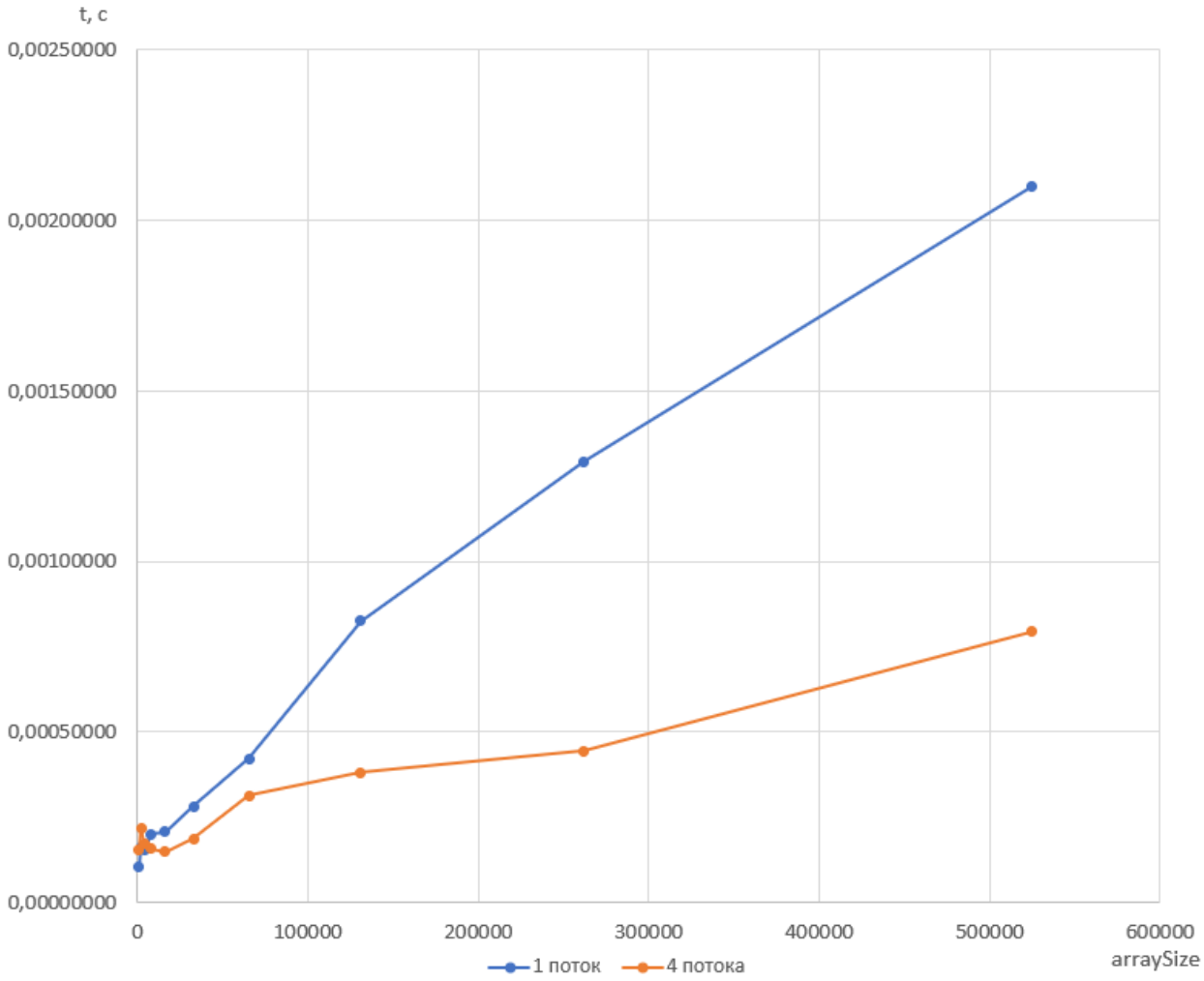


Рис. 2 – Зависимость времени выполнения сложения векторов от их размера на 1 и 4 потоках CPU.

Из полученных данных можно сделать вывод, что параллельное сложение векторов на CPU происходит быстрее, чем последовательное при длине

векторов больше 8000. Преимущество по времени более чем в 2 раза наблюдается при длине векторов больше 130 000.

2.

Для определения оптимального количество потоков POSIX для распараллеливания напишем программу, которая будет выполнять сложение векторов используя разное количество потоков, по полученным результатам построим график.

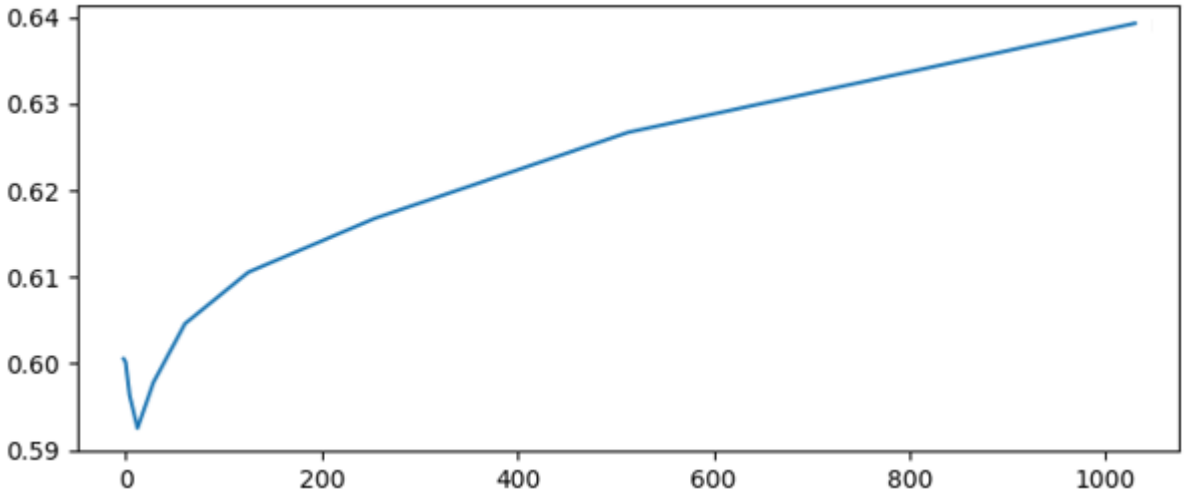


Рис. 3 – Зависимость времени выполнения сложения векторов от количества потоков CPU.

Из зависимости видно, что оптимального количество потоков POSIX для распараллеливания в нашем случае равно 4.

3.

Напишем программу для определения зависимости времени выполнения операции сложения на GPU от длины векторов (количество нитей равно длине вектора). По полученным данным построим график:

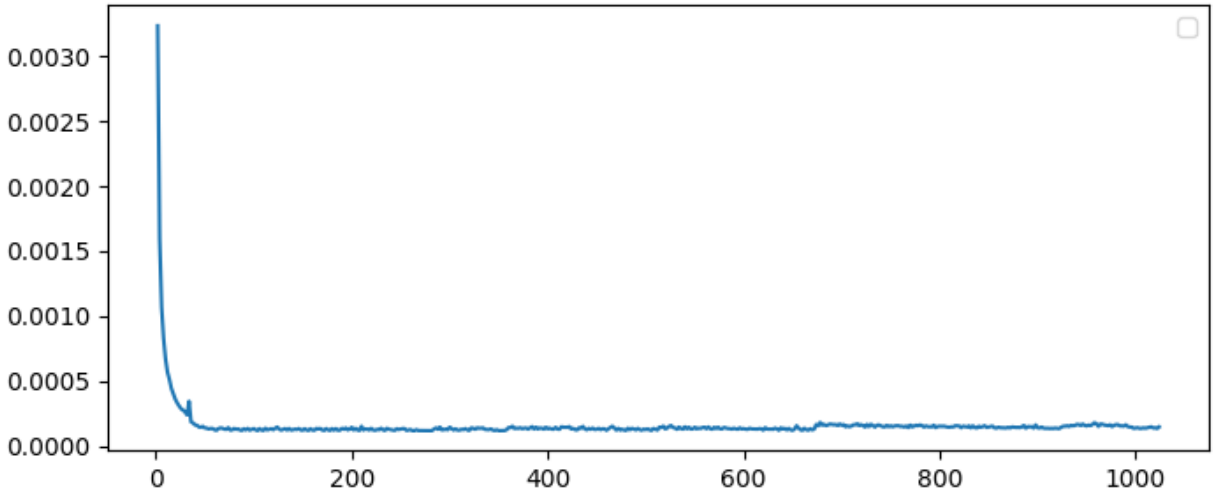


Рис. 4 – Зависимость времени выполнения сложения векторов от количества потоков GPU.

Вывод:

- Мы определили при какой длине векторов имеет смысл распараллеливать операцию сложения, используя потоки CPU (от 130 000 эл.), параллельное сложение векторов на GPU происходит быстрее, чем последовательное при любой длине векторов (разница во времени более чем в 2 раза наблюдается при длине векторов больше 128).
- Определили оптимальное количество потоков POSIX для распараллеливания (4).
- Построили график зависимость времени выполнения операции сложения на GPU от длины векторов (количество нитей равно длине вектора).
- Получили начальные знания распараллеливания кода на GPU.

Листинг

1.

main.cu (gpu)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <chrono>
#include <cuda.h>
#include <math.h>
#include <stdio.h>
#include <iostream>
#include <fstream>
#include <iomanip>

__global__ void addKernel(int* c, const int* a, const int* b, int size) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = tid; i < size; i += stride) {
        c[i] = a[i] + b[i];
    }
}

int addWithCUDA(int* c, const int* a, const int* b, int size, int gridSize, int
blockSize) {
    int* dev_a = nullptr, * dev_b = nullptr, * dev_c = nullptr;

    cudaMalloc((void**)&dev_a, size * sizeof(int));
    cudaMalloc((void**)&dev_b, size * sizeof(int));
    cudaMalloc((void**)&dev_c, size * sizeof(int));

    cudaMemcpy(dev_a, a, size * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(dev_b, b, size * sizeof(int), cudaMemcpyHostToDevice);

    // Последовательное
    auto start = std::chrono::steady_clock::now();
    addKernel <<<1, 1>>> (dev_c, dev_a, dev_b, size);
    cudaDeviceSynchronize();
    auto endSingle = std::chrono::steady_clock::now();
    std::chrono::duration<double> singleDuration = endSingle - start;

    // Параллельное
    start = std::chrono::steady_clock::now();
    addKernel <<<gridSize, blockSize>>> (dev_c, dev_a, dev_b, size);
    cudaDeviceSynchronize();
    auto endMulti = std::chrono::steady_clock::now();
    std::chrono::duration<double> multiDuration = endMulti - start;

    // Расчет коэффициента (отношение времен выполнения)
    double speedupFactor = singleDuration.count() / multiDuration.count();

    cudaMemcpy(c, dev_c, size * sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(dev_a);
    cudaFree(dev_b);
    cudaFree(dev_c);

    std::ofstream outputFile("results.txt", std::ios::app);
    if (outputFile.is_open()) {
        int testInd = size * 0.6;
        outputFile << size << "|" << std::fixed << std::setprecision(8) <<
singleDuration.count() << ", " << multiDuration.count() << ", " << speedupFactor <<
"|" << c[testInd] << ", " << a[testInd] << ", " << b[testInd] << std::endl;
```

```
        outputFile.close();
        if (singleDuration.count() >= 1) {
            std::cerr << "Время выполнения больше 1 сек, выыполнение программы
остановлено." << std::endl;
            return -1;
        }
    }
    else {
        std::cerr << "File ERROR" << std::endl;
        return -2;
    }

    return 0;
}

int main() {
    system("chcp 1251");
    const int maxSize = 1024*1024;
    const int blockSize = 256;
    std::ofstream outputFile("results.txt");

    if (outputFile.is_open()) {
        outputFile.close();
    }
    else {
        std::cerr << "File ERROR" << std::endl;
        return 1;
    }

    for (int arraySize = 2; arraySize <= maxSize; arraySize *= 2) {
        int* a, * b, * c;
        a = new int[arraySize];
        b = new int[arraySize];
        c = new int[arraySize];

        // Инициализация векторов случайными значениями
        for (int i = 0; i < arraySize; ++i) {
            a[i] = rand();
            b[i] = rand();
        }

        int gridSize = (arraySize
            + blockSize - 1) / blockSize;

        if (addWithCUDA(c, a, b, arraySize, gridSize, blockSize) != 0)
            return 0;

        delete[] a;
        delete[] b;
        delete[] c;
    }

    return 0;
}
```

main.cpp (cpu)

```
#include <iostream>
#include <iomanip>
#include <chrono>
#include <pthread.h>
#include <cstdlib>
#include <fstream>
```

```
#define MAX_SIZE 1024*1024
#define NUM_THREADS 4

const int* a, * b;
int* c;
int arraySize;

void* add(void* thread_id) {
    long tid = (long)thread_id;
    int chunk_size = arraySize / NUM_THREADS;
    int start = tid * chunk_size;
    int end = (tid == NUM_THREADS - 1) ? arraySize : start + chunk_size;

    for (int i = start; i < end; ++i) {
        c[i] = a[i] + b[i];
    }

    pthread_exit(NULL);
}

double addWithPthreads(int* result, const int* arr_a, const int* arr_b, int size)
{
    a = arr_a;
    b = arr_b;
    c = result;
    arraySize = size;

    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    void* status;

    auto start = std::chrono::steady_clock::now();

    for (t = 0; t < NUM_THREADS; ++t) {
        rc = pthread_create(&threads[t], NULL, add, (void*)t);
        if (rc) {
            std::cerr << "Error: unable to create thread, " << rc << std::endl;
            exit(-1);
        }
    }

    for (t = 0; t < NUM_THREADS; ++t) {
        rc = pthread_join(threads[t], &status);
        if (rc) {
            std::cerr << "Error: unable to join thread, " << rc << std::endl;
            exit(-1);
        }
    }
```

```
    auto end = std::chrono::steady_clock::now();
    std::chrono::duration<double> duration = end - start;

    return duration.count();
}

int main() {
    std::ofstream outputFile("results_pthreads.txt");

    if (!outputFile.is_open()) {
        std::cerr << "File ERROR" << std::endl;
        return 1;
    }

    for (int arraySize = 2; arraySize <= MAX_SIZE; arraySize *= 2) {
        int* a = new int[arraySize];
        int* b = new int[arraySize];
        int* c = new int[arraySize];

        // Инициализация векторов случайными значениями
        for (int i = 0; i < arraySize; ++i) {
            a[i] = rand();
            b[i] = rand();
        }

        double duration = addWithPthreads(c, a, b, arraySize);
        outputFile << std::fixed << std::setprecision(8) << duration <<
std::endl;

        delete[] a;
        delete[] b;
        delete[] c;
    }

    outputFile.close();
    return 0;
}
```

2.

main.cpp (CPU)

```
#include <chrono>
#include <math.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct Indexes {
  unsigned int start_index;
```

```
  unsigned int end_index;
} Indexes;

float *A;
float *B;
float *C;

void *sum(void *arg) {
  Indexes indexes = *(Indexes *)arg;
  unsigned int start_index = indexes.start_index;
  unsigned int end_index = indexes.end_index;
  for (unsigned int i = start_index; i < end_index; i++) {
    C[i] = A[i] + B[i];
  }
  return 0;
}

int main() {
  unsigned long long int N = pow(2, 28);
  FILE *fp = fopen("data_posix.txt", "w");
  for (int threads = 2; threads <= 1024; threads *= 2) {
    A = (float *)calloc(N, sizeof(float));
    B = (float *)calloc(N, sizeof(float));
    C = (float *)calloc(N, sizeof(float));
    for (unsigned long long int i = 0; i < N; i++) {
      A[i] = 1;
      B[i] = 1;
      C[i] = 0;
    }
    unsigned long long int items_per_threads = N / threads;
    Indexes indexes[threads];

    for (int i = 0; i < threads; i++) {
      Indexes index;
      index.start_index = i * items_per_threads;
      index.end_index = index.start_index + items_per_threads;
      indexes[i] = index;
    }

    auto start = std::chrono::high_resolution_clock::now();
    pthread_t threads_array[threads];
    for (int i = 0; i < threads; i++) {
      pthread_create(&threads_array[i], NULL, sum, &indexes[i]);
    }
    for (int i = 0; i < threads; i++) {
      pthread_join(threads_array[i], NULL);
    }
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<long double> difference = end - start;
    printf("%d|%Lf\n", threads, difference.count());
```

```
    fprintf(fp, "%Lf %d\n", difference.count(), threads);
    free(A);
    free(B);
    free(C);
  }
}
```

3.

main.cu (GPU)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <chrono>
#include <cuda.h>
#include <math.h>
#include <stdio.h>
#include <iostream>
#include <fstream>
#include <iomanip>

__global__ void addKernel(int* c, const int* a, const int* b, int size) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = tid; i < size; i += stride) {
        c[i] = a[i] + b[i];
    }
}

int addWithCUDA(int* c, const int* a, const int* b, int size, int gridSize, int
blockSize) {
    int* dev_a = nullptr, * dev_b = nullptr, * dev_c = nullptr;

    cudaMalloc((void**)&dev_a, size * sizeof(int));
    cudaMalloc((void**)&dev_b, size * sizeof(int));
    cudaMalloc((void**)&dev_c, size * sizeof(int));

    cudaMemcpy(dev_a, a, size * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(dev_b, b, size * sizeof(int), cudaMemcpyHostToDevice);

    auto start = std::chrono::steady_clock::now();
    addKernel <<<gridSize, blockSize >>> (dev_c, dev_a, dev_b, size);
    cudaDeviceSynchronize();
    auto endMulti = std::chrono::steady_clock::now();
    std::chrono::duration<double> multiDuration = endMulti - start;

    cudaMemcpy(c, dev_c, size * sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(dev_a);
    cudaFree(dev_b);
    cudaFree(dev_c);

    std::ofstream outputFile("results.txt", std::ios::app);
    if (outputFile.is_open()) {
        outputFile << std::fixed << std::setprecision(8) << multiDuration.count() <<
" " << size << std::endl;
        outputFile.close();
    }
    else {
        std::cerr << "File ERROR" << std::endl;
        return -2;
    }
```

```
    return 0;
}

int main() {
    system("chcp 1251");
    const int maxSize = 1024;
    std::ofstream outputFile("results.txt");

    if (outputFile.is_open()) {
        outputFile.close();
    }
    else {
        std::cerr << "File ERROR" << std::endl;
        return 1;
    }

    for (int arraySize = 2; arraySize <= maxSize; arraySize += 2) {
        int* a, * b, * c;
        a = new int[arraySize];
        b = new int[arraySize];
        c = new int[arraySize];

        // Инициализация векторов случайными значениями
        for (int i = 0; i < arraySize; ++i) {
            a[i] = rand();
            b[i] = rand();
        }

        int gridSize = (arraySize
            + arraySize - 1) / arraySize;

        if (addWithCUDA(c, a, b, arraySize, gridSize, arraySize) != 0)
            return 0;

        delete[] a;
        delete[] b;
        delete[] c;
    }

    return 0;
}
```